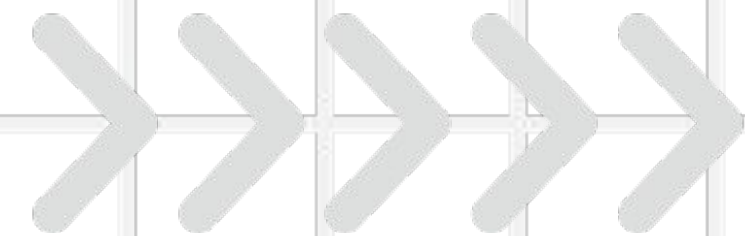


# ALGORITMOS DE ORDENAÇÃO

Introdução a Programação | 2026.1



Engenharia de Software

**INF**  
INSTITUTO DE  
INFORMÁTICA



**UFG**  
UNIVERSIDADE  
FEDERAL DE GOIÁS

# O QUE É UM ALGORITMO DE ORDENAÇÃO?

Os algoritmos de ordenação são um conjunto de instruções que recebem um array ou lista como entrada e organizam os itens em uma ordem específica.

As ordenações mais comumente realizadas são a numérica ou a alfabética (também chamada de lexicográfica) e podem ser em ordem crescente (A-Z, 0-9) ou decrescente (Z-A, 9-0).



# **POR QUE SÃO IMPORTANTES?**

A ordenação pode, muitas vezes, reduzir a complexidade de um problema, pois:

- Aceleram buscas ao organizar dados.
- Melhoram desempenho de algoritmos subsequentes.
- Reduzem requisitos de memória.

Além da organização de dados, algoritmos de ordenação são fundamentais em áreas como algoritmos de grafos, aprendizado de máquina, entre outros.

# **ALGORITMOS DE ORDENAÇÃO COMUNS**

Alguns dos algoritmos de ordenação mais comuns são:

- Selection Sort (ordenação por seleção)
- Bubble Sort
- Insertion Sort (ordenação por inserção)
- Merge Sort (ordenação por intercalação)
- Quick Sort (ordenação rápida)
- Heap Sort
- Counting Sort
- Radix Sort
- Bucket Sort

# **CLASSIFICAÇÃO DE UM ALGORITMO DE ORDENAÇÃO**

**Qual o tamanho da coleção que está sendo ordenada?  
Quanta memória está disponível para uso? A coleção precisa crescer?**

**As respostas a essas perguntas podem determinar qual  
algoritmo funcionará melhor para a situação.**



# **CLASSIFICAÇÃO DE UM ALGORITMO DE ORDENAÇÃO**

Os algoritmos de ordenação podem ser categorizados com base nos seguintes parâmetros:

- Número de trocas ou inversões necessários;
- Número de comparações;
- Uso ou não de recursão;
- Estabilidade;
- Necessidade de espaço adicional.



# BUBBLE SORT

```
func bubbleSort(arr []int) {  
    n := len(arr)  
    for i := 0; i < n-1; i++ {  
        for j := 0; j < n-i-1; j++ {  
            if arr[j] > arr[j+1] {  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
            }  
        }  
    }  
}
```

# SELECTION SORT

```
func selectionSort(arr []int) {  
    n := len(arr)  
    for i := 0; i < n-1; i++ {  
        minIndex := i  
        for j := i + 1; j < n; j++ {  
            if arr[j] < arr[minIndex] {  
                minIndex = j  
            }  
        }  
        arr[i], arr[minIndex] = arr[minIndex], arr[i]  
    }  
}
```

# INSERTION SORT (ORDENAÇÃO POR INSERÇÃO)

```
func insertionSort(arr []int) {  
    n := len(arr)  
    for i := 1; i < n; i++ {  
        key := arr[i]  
        j := i - 1  
        for j >= 0 && arr[j] > key {  
            arr[j+1] = arr[j]  
            j--  
        }  
        arr[j+1] = key  
    }  
}
```

# MERGE SORT

```
func mergeSort(arr []int, l, r int) {  
    if l >= r {  
        return  
    }  
    m := (l + r) / 2  
    mergeSort(arr, l, m)  
    mergeSort(arr, m+1, r)  
    merge(arr, l, m, r)  
}
```

```
func merge(arr []int, l, m, r int) {  
    n1 := m - l + 1  
    n2 := r - m  
  
    left := make([]int, n1)  
    right := make([]int, n2)  
    copy(left, arr[l:m+1])  
    copy(right, arr[m+1:r+1])  
  
    i, j, k := 0, 0, l  
    for i < n1 || j < n2 {  
        if j == n2 || (i < n1 && left[i] <= right[j]) {  
            arr[k] = left[i]  
            i++  
        } else {  
            arr[k] = right[j]  
            j++  
        }  
        k++  
    }  
}
```

# ANIMAÇÕES



# REFERÊNCIAS



- 🔍 SORTING ALGORITHMS EXPLAINED WITH EXAMPLES IN JAVASCRIPT, PYTHON, JAVA, AND C++  
Disponível em:  
<https://www.freecodecamp.org/news/sorting-algorithms-explained-with-examples-in-python-java-and-c/>
- 🔍 ALGORITMOS DE ORDENAÇÃO EM GO  
Disponível em:  
[https://github.com/Henriquemcc/Algoritmos\\_de\\_ordenacao\\_em\\_Go/tree/main](https://github.com/Henriquemcc/Algoritmos_de_ordenacao_em_Go/tree/main)

# REFERÊNCIAS

🔍 Algoritmos de Busca - Algoritmos e Programação de Computadores, Zaroni Dias  
Disponível em:  
<https://ic.unicamp.br/~mc102/aulas/aula10.pdf>

# Dúvidas?

